90-day engineering plan

PART A / Regulated transactional platform / Diagnosis, priorities and execution

Launch region 4 in week 10 with demonstrable money controls. By day 90, make delivery and recovery less dependent on individual heroics.

The common root cause is quality and operational ownership concentrated in too few people, too late. Duplicate credits and float-based reporting show that money rules are not consistently enforced by the system. Three smaller payment-adjacent incidents passed review: more review alone cannot fix missing invariants and failure tests.

Two QA supporting roughly 20 engineers, three live regions and about 30 brands cannot be the sole checkpoint for every feature and configuration. The ratio alone does not establish the required staffing, but late regressions and slipping releases demonstrate an overloaded model. Incidents consume capacity, releases accumulate, and QA receives larger batches with less time to explore.

S. and T. are critical knowledge dependencies. Reported review behaviour may discourage questions; pipeline stalls during T.'s leave demonstrate inadequate backup capability. Rising MTTR and informal response suggest weak recovery ownership. The region-3 overrun and region-2 configuration error suggest launch plans omit dependencies and operational readiness; I will test that diagnosis through the missing retrospective.

Days 1-14: contain exposure and establish the real plan. Verify previous fixes across live regions and assess historical money exposure with Finance. Name incident responders and backups. Reconstruct region-3 delivery. Agree minimum launch scope, regulatory acceptance criteria, provider certification dates and dependency owners. Produce an evidence-based forecast by day 14.

Days 15-42: build controls and integrations together. Prioritise atomic idempotency, safe handling of uncertain withdrawals, exact monetary arithmetic and reconciliation. Bound batch concurrency and database connection usage. Automate regional configuration validation. Required providers must demonstrate deposit, withdrawal, timeout and retry flows; a backup must recover the pipeline without T.

Days 43-70: rehearse, close gaps and launch. Complete end-to-end acceptance, load tests and production-like deployment/recovery rehearsals by week 8. Verify provider account identity and credentials. Resolve critical blockers, then launch with limited exposure where permitted and expand against agreed stop criteria.

Days 71-90: consolidate. Monitor through settlement cycles, complete the launch retrospective, close high-risk prevention actions and make the regional launch procedure reusable. Demonstrate independent wallet and pipeline backup ownership before restoring deferred commitments.

Explicit trade-offs: no wallet rewrite, microservice migration, squad reorganisation, broad automation programme, discretionary CRM expansion or cosmetic web roadmap this quarter. Defer low-impact bugs; retain mandatory reporting and serious existing-customer fixes. AI making a low-priority task easier does not make it a priority.

Recovery must preserve financial correctness. Application rollback does not undo financial transactions. Containment must preserve records and resolve in-flight payments, delayed callbacks and audited corrections. After launch, retain ownership until settlement and reconciliation confirm the outcome.

Change how work reaches production

PART A / Three operating changes and four health signals

1. Define behaviour before coding; developers own automated proof. Product, QA and the developer agree Given/When/Then examples before implementation. Start BDD with deposits, withdrawals, wallet adjustments and regional configuration. QA leads test design; developers implement and run the agreed automated tests before QA handover. QA protects time for exploratory testing, unexpected combinations and independent validation.

Happy paths are necessary but insufficient. Duplicate callbacks, concurrent withdrawals, delayed notifications and a timeout after provider acceptance are mandatory developer-owned acceptance scenarios. Finance or the domain owner validates expected monetary outcomes. Success means less work returned from QA, shorter QA waiting time and fewer serious escaped regressions.

2. Review according to risk, with AI assistance. Money-impacting changes receive two human review rounds: implementation and test quality first; domain correctness, concurrency, retries and recovery second. A qualified domain reviewer must be independent of the author. Grow reviewers beyond S. Routine low-risk changes receive one human review plus automated checks, avoiding a second universal queue.

Document conventions and use AI to flag deviations, draft acceptance tests and assist with bounded maintenance. Use deterministic linting, type checks and tests wherever possible. Developers verify all generated changes; AI is neither an independent financial approver nor proof of correctness. Trial the approach for two weeks and compare total completion time, review effort, rework and defects. Count savings only when demonstrated.

3. Establish one operating rhythm for incidents and priorities. Maintain primary/backup coverage. Serious incidents have a commander, technical responder and communications owner; suspected money corruption immediately involves Finance. Review within two working days of stabilisation and fund owned prevention actions. Each week, Engineering, Product, QA and dependency owners review launch evidence, blockers and interruption load. Urgent work explicitly displaces another commitment.

Success means faster detection and mitigation, fewer repeated failure mechanisms, backup-led recovery drills and earlier escalation of launch blockers. Smaller releasable changes reduce QA batch size; I will not impose weekly releases without the controls to support them.

Baseline the following signals in the first fortnight. Review trends by service or region, use existing records, and connect each worsening signal to a decision.

Financial correctness: confirmed incidents and gross affected value, unexplained reconciliation exception age, and reconciliation freshness. This reveals both harm and detection blind spots. Recovered losses remain visible; expected settlement delays are distinguished from unexplained discrepancies.

Recovery: time to detect and mitigate serious incidents, separating service restoration from final financial correction. This tests operational readiness. With small samples, review individual cases alongside the trend.

Flow with stability: median and 85th-percentile change lead time, identifying QA waiting time, paired with the proportion of releases needing rollback or an urgent fix. Faster delivery only counts as improvement if stability holds.

Unplanned work: a lightweight weekly estimate of capacity spent on incidents, support and rework. This explains forecast reliability and whether prevention is returning usable capacity. Do not use these measures to rank individuals.

Make the plan achievable

PART A / People, capacity and launch accountability

Allocate named capacity, not theoretical headcount. The roster lists 18 engineers despite the headline saying 20; resolve this immediately. Provisionally assign S., one mid and one junior to safety work, and two mids plus two juniors to integrations. Both streams retain operational duties. CRM prioritises mandatory reporting and pipeline resilience; Web prioritises essential regional journeys and support tooling. Keep squads intact.

Initially reserve 20% of capacity for support and incidents, then adjust from observed demand. Include pairing, reviews and management time in estimates. If interruptions exceed the reserve, revise scope and forecast immediately rather than silently borrowing evenings or safety capacity.

S. needs clear behavioural expectations and a knowledge-transfer challenge. Validate the predecessor's report through confidential conversations and specific examples. Require respectful, actionable reviews immediately; coach and follow up, using the normal performance process if harmful behaviour continues. Pair S. with a mid on wallet work and drills, with the mid increasingly leading. Another capable wallet owner is part of S.'s delivery outcome.

D. and the other mids need bounded ownership; juniors need support. Give mids ownership of integrations and controls with scheduled technical guidance. Assess readiness before selecting a launch technical lead. Give each junior a named pairing partner and bounded tasks, with safe routes to raise uncertainty. Juniors contribute to critical work without becoming sole approvers or unsupported responders.

T. needs reduced scope and protected transfer time. Remove enough backlog responsibility to train a pipeline backup and rehearse restart/replay procedures. Documentation and recovery drills are funded delivery work. Confirm transfer through a recovery exercise without T.'s help, not just the existence of a runbook.

L. needs recognition and limits on invisible work. Agree a bounded mentoring contribution and remove equivalent delivery scope. Do not turn a reliable lead into the organisation's unofficial people manager; I remain accountable for cross-squad coaching and performance issues.

QA needs earlier involvement and relief from sole ownership of regression testing. One QA primarily partners with safety work, the other with launch acceptance. Engineers automate agreed happy paths and critical failure scenarios; QA leads test design and exploration. Measure residual workload before deciding whether more QA capacity is necessary. BDD redistributes responsibility; it does not eliminate QA work.

Keep recruiting while testing AI-assisted capacity. In week one, investigate compensation, role clarity, sourcing and interview delays behind the three-month vacancies. Prioritise a senior payments/backend engineer and a backend engineer with strong platform/data operational experience. Hire for judgement and constructive collaboration. Use approved AI tooling for bounded tasks with human verification; neither new hires nor assumed AI savings enter the week-10 critical path.

Launch accountability is explicit. Agree with the CEO by day 14 which brands, providers, journeys and volumes constitute launch. Compliance confirms required permissions and acceptance; Finance accepts reconciliation and reporting; Engineering demonstrates verified configuration, trained coverage and containment. No unresolved critical money defect or unexplained financial discrepancy passes the gate.

Zero money-impacting incidents is the objective, not a promise that failure is impossible. If evidence threatens week 10, escalate early with scope options. If no compliant, financially safe scope fits, recommend delaying real-money activation.

Cashback code review

PART B / Request changes / Financial correctness, scale and operations

References: continuous lines across the entire pasted diff, including headers/blanks; no restart by file/function. Static review; application tests not run. File keys below retain exact paths.

S src/services/cashbackService.ts | **M** src/db/migrations/20260710000003-create-cashback-payouts.js
D src/db/models/cashbackPayout.ts | **R** src/routes/admin.ts | **A** src/app.ts | **T** test/cashback.test.ts

BLOCKING - S:90-106; M:22; D:44,53
parseFloat()/JavaScript numbers and FLOAT storage repeat incident 4's precision failure. Given the supplied DECIMAL(36,18)/dec() convention, use dec() for rate, losses, cashback and balance; apply currency rounding; change migration/model to DECIMAL(36,18) and align the amount type. Storage changes alone cannot recover lost precision.

BLOCKING - S:106-119
Separate balance, payout and ledger writes leave partial credits when an insert fails. Use the existing wallet service with one short transaction per payout; pass it to every write and catch errors outside the callback to trigger rollback. One transaction for the entire run would hold locks/connections too long.

BLOCKING - S:101-107
Read-modify-save can erase concurrent activity: cashback reads 100, a deposit makes 150, then cashback saves 105. Use the wallet engine's atomic update or locking/version protocol across all writers. Transactions alone do not protect stale reads; concurrency protection is separate from idempotency.

BLOCKING - S:109-113; M:11-26
Retries/overlapping runs can pay twice. Define a stable member/period/account/programme/currency payout identity as applicable; enforce a database unique constraint and claim it atomically with the credit. Check-before-insert races; job locks alone do not ensure safe crash recovery.

BLOCKING - S:75-78
accountId is ignored; Member.findAll() selects every tenant unless unseen scoping prevents it. Filter by the authorised account and retain ownership checks through bets, wallets and payout auditing. UUID validation is not permission.

BLOCKING - S:76,86-88
Now-minus-seven-days changes on retries, and no upper bound excludes bets arriving during processing. Resolve a completed canonical period in the programme timezone; persist/reuse [periodStart, periodEnd) for calculation, retries and deduplication. Processing order must not change eligibility.

BLOCKING - S:86-92
Eligibility/settlement is unchecked: a pending bet with zero payout becomes a realised loss. Confirm the lifecycle; filter eligible settled outcomes and define the period timestamp, void/refund rules, late settlements and corrections so calculations are reproducible.

MAJOR - S:68,76 (configuration)
Hard-coded CASHBACK_RATE and 7 * 24 * 60 * 60 * 1000 require code changes to alter policy. Load validated CASHBACK_RATE and CASHBACK_PERIOD_DAYS from environment config; parse the rate with dec(), enforce approved rate bounds and positive bounded integer days, and reject invalid values. Centralise period calculation; preserve timezone-aware calendar boundaries. Snapshot settings per run for safe retries; support approved brand overrides where policies differ. Environment config alone does not fix rolling windows.

VERIFY - S:86-92,101 (blocking if applicable)
If members have multiple currencies/wallet types, summing all bets and finding any member wallet can credit the wrong one. Confirm schema guarantees; otherwise calculate by currency/programme, select the matching wallet and record that identity.

VERIFY - R:145-149; A:168
No authentication/account permission is visible; global middleware may cover it. Confirm administrative authorisation for the requested account, audit the actor, and test unauthenticated, insufficient-permission and cross-account requests. Missing protection blocks release.

BLOCKING - S:78-125; R:145-149
At 40,000 members x 5 brands, the HTTP run loads 200,000 members, then issues one bet query each plus a wallet lookup/three writes per successful payout: 200,001 + 4 x eligible members, up to 1,000,001 operations. An illustrative 5 ms each means about 83 minutes before other overhead, not a benchmark. Timeouts/restarts leave partial work; five sequential brand requests can repay the entire population.

=> Scope the brand; enqueue a durable job and return 202/run ID. Use keyset pagination, bounded bet aggregation, indexed filters, bounded workers and short payout transactions. Persist resumable progress; prevent overlapping brand/programme/period jobs and retain payout-level idempotency. A 500-member page gives 80 pages/brand: load-test it. Avoid full-population Promise.all(). One serial run does not itself exhaust the pool; overlapping runs compete with live payments.

BLOCKING - S:102-103,122-127; R:148-149
Errors are swallowed and missing wallets silently skipped, yet ok: true can report total failure. Persist run/member outcomes: paid, already-paid, ineligible or failed. Expose partial failure; retry with original periods and payout identities.

MAJOR - S:75-127, especially 123
console.log is the only explicit failure reporting; automatic APM/log collection is unverified. Follow logger/APM conventions: correlate run/account/period/payout IDs; capture duration, progress, outcomes and error details; alert on failed/stalled runs and database latency. Keep durable financial audit records separately: traces can be sampled or expire.

REQUIRED - T:181-187,190-215
Mocks assert calls, not the 3.00 credit, 13.00 balance or matching payout/ledger amounts. Assert exact results; add real-database tests for sequential/concurrent duplicates, failure after every write, concurrent deposits/withdrawals, tenant/currency isolation, period boundaries, eligible states, rounding and partial-run recovery. Test API permissions and config validation/snapshot reuse; load-test 200,000 members alongside live traffic. Mocks cannot prove uniqueness, rollback or concurrency.

The conversation

PART B / S. approved the change; D. authored it

I would block the merge on the confirmed money-safety defects and first establish whether any version is already live. If it is, containment and reconciliation take priority over the review conversation.

With D., privately: "You own this implementation, and these failure paths need fixing. Walk me through the requirements, tests, and assumptions you were working with." I would establish whether D. knew the wallet APIs, money conventions, and retry requirements, and whether deadline pressure or review dynamics discouraged questions. S.'s approval does not remove D.'s authorship responsibility, but I would not leave D. to repair unfamiliar financial code alone. D. owns the correction and regression tests, paired with a qualified engineer.

With S., privately: "Your approval gave the team confidence this was safe. Help me understand what you checked and why these risks were missed." I would distinguish a knowledge gap, unclear review scope, overload, and an inadequate review. Staff-level responsibility includes making reasoning visible and developing others. If D. felt unable to challenge S., I would address that behaviour explicitly, without exposing confidential comments. One missed review warrants learning and clear expectations; repeated careless approval or harmful behaviour warrants performance action.

Then together: Agree the failure mechanisms, correction owners, and acceptance evidence. Keep the discussion factual and avoid turning D. into the scapegoat or S. into the mandatory reviewer for everything.

To prevent recurrence: Make financial safety part of the normal delivery path:

- Route credits through a shared wallet API enforcing atomicity, concurrency protection, and idempotency.
- Add automated acceptance tests for retries, partial failures, tenant isolation, and exact arithmetic.
- Require qualified domain review for money changes, with explicit evidence rather than an approval checkbox.
- Train a second wallet reviewer and budget review time.

I would sample the next few changes and track escaped defects and review delays, then step back.

My role is to establish a dependable review system, not become its permanent final checkpoint.

Two situations

PART C / Incident leadership and a public challenge

C1. Saturday, 01:40: balances are increasing.

01:40-01:50: I declare a critical incident and take command. To the CEO: "Potentially. Reported balances are increasing unexpectedly; we have not verified cash loss or the cause. We are treating exposure as ongoing and containing it now. Next update at 02:00." I open one incident channel and decision log.

S. leads diagnosis, paired with D. to challenge actions and record findings. The on-call engineer executes containment; Support owns customer updates; Finance quantifies exposure; Compliance assesses notification obligations. I coordinate decisions and communications rather than guess at unfamiliar code. T. stays on leave; use the pipeline backup.

First hour: Hold region-2 withdrawals and other ways to spend or transfer suspect balances. Pause the suspect posting path while durably retaining incoming callbacks for controlled replay. If scope cannot be isolated quickly, suspend regional money movement. Verify that containment stopped new incorrect postings; track in-flight payouts with providers. Preserve logs, ledger records and configuration history; freeze unrelated regional changes. Check other regions for the same signature.

Hours 1-6: S. and D. distinguish display errors from ledger corruption and reproduce the failure. Finance reconciles provider records, payments and ledger entries, reporting affected accounts, gross erroneous credits and actual funds paid out separately. Give the CEO and Support updates every 30 minutes until contained, then hourly, including unknowns and the next decision.

Hours 6-24: Independently review the fix; QA validates the failure scenario and reconciliation evidence. Restore gradually only after incorrect postings cease and uncertain transactions are accounted for. Rollback alone cannot reverse money movements; corrections require Finance-approved, auditable entries. Arrange relief shifts, customer remediation and Compliance-led notifications where required. By 24 hours, publish verified impact, remaining restrictions, owners and the review schedule.

C2. Week 6: L. challenges the release checklist.

In the next 60 seconds, I pause and respond calmly: "L., if the checklist adds delay without reducing risk, we should change it. I own that. But we cannot ask teams to ignore agreed release controls. Which item is blocking this sprint, and what would provide the same safety evidence with less work? We will capture it now, continue planning against the current controls, and resolve it today."

I hear one concrete example, check whether it blocks today's planning, and assign a same-day review with L. and QA. I neither demand a public apology nor turn the meeting into a debate about my authority.

After the meeting, I speak privately with L. I ask what prompted the challenge, what burden the team is experiencing, and whether my rollout ignored earlier feedback. I explain the impact: a respected lead telling teams to bypass controls creates conflicting instructions and leaves engineers carrying unmanaged risk. Challenging the plan is welcome; directing unilateral noncompliance is not.

Together with QA and a developer, we inspect real examples and waiting times. Remove duplicate approvals, automate evidence collection and simplify low-risk requirements. Retain money-correctness evidence and applicable compliance gates. Fund L.'s contribution by removing equivalent work. If essential controls threaten the date, take a scope trade-off to the CEO.

That day, I publish the decision to both squads, acknowledge any mistake in the original process, and ask L. to reinforce the agreed approach. Time-box changes to a one-week trial, measuring review delay, QA rework and escaped defects. Follow up privately on trust and workload; repeated deliberate bypass after clear expectations becomes a performance issue, not punishment for dissent.